Merging Five GLUE NLP Tasks Using Weight-Ensembling Mixture of Experts

Few-Shot Router Training on a Shared BERT Backbone

Research Report

AUTHORS (GROUP 1):

NGUYỄN BÁ ANH 20261218M

NGUYỄN THỊ NHÃ LINH 20261262M

INSTRUCTOR:

NGUYỄN KIÊM HIẾU

Hanoi University of Science and Technology

CONTENTS

Abstract **2**

Chapter 1. The Problem **3**
1.1 Why One Separate Model per Task Does Not Scale . . . 3
1.2 Problem Statement and Research Questions . . . 3
1.3 Why the Problem Matters and Why It Is Difficult . . . 4
1.4 Why Static Merging Alone May Be Insufficient . . . 4

Chapter 2. The Idea **5**
2.1 Mixture of Experts: The Core Concept . . . 5
2.2 From Output Ensembling to Weight Ensembling . . . 5
2.3 Implementation Flow: From Datasets to Evaluation . . . 6

Chapter 3. Theory and Method: WEMoE **7**
3.1 BERT Encoder and Shared Backbone . . . 7
3.2 WEMoE Architecture: Static and Input-Conditioned Merging . . . 7
3.3 Task Arithmetic and Task Vectors . . . 9
3.4 Router and Input-Conditioned MLP . . . 9
3.5 Few-Shot Fine-Tuning (FSFT) Objective . . . 10

Chapter 4. Experimental Methodology **11**
4.1 The Five Binary GLUE Tasks and Their Datasets . . . 11
4.2 Data Splits and Experimental Protocol . . . 11
4.3 Training the Five Task-Specific Experts . . . 11
4.4 Merging the Experts and Training the Router with Few-Shot Data . . . 12
4.5 Evaluation Metrics, Baselines, Seeds and Reproducibility . . . 13

Chapter 5. Results **14**
5.1 Results by Task . . . 14
5.2 Comparison with Relevant Baselines . . . 15
5.3 Routing Behaviour . . . 16
5.4 Storage, Parameter Count and Inference Latency . . . 17

Chapter 6. Trade-offs and Discussion **18**
6.1 What the Results Demonstrate . . . 18
6.2 Accuracy, Parameter/Storage and Latency Trade-offs . . . 18
6.3 Practical Implications . . . 18
6.4 Limitations, Threats to Validity and Unresolved Questions . . . 18
6.5 Demonstrated Findings and Open Hypotheses . . . 19

Chapter 7. Conclusion and Future Work **20**
7.1 Answers to the Research Questions . . . 20
7.2 Assessment of the Objectives . . . 20
7.3 Main Contributions and Practical Implications . . . 20
7.4 Future Work . . . 21

References **22**

Abstract

Fine-tuning one pre-trained language model per task gives strong task-specific performance, but every new task adds a complete model to train, store and serve, and nothing learned for one task reaches the others. This study examines whether several fine-tuned models can become a single multi-task model whose weights adapt to the input. We adapt Weight-Ensembling Mixture of Experts (WEMoE), previously evaluated on vision Transformers, to BERT and five binary tasks of the GLUE benchmark: MRPC, RTE, CoLA, SST-2 and QNLI. Weights outside the MLP blocks are merged statically with Task Arithmetic, while the MLP weights of every encoder layer are rebuilt for each input as the pre-trained weights plus a router-weighted sum of frozen task vectors. Only the routers are trained, by Few-Shot Fine-Tuning (FSFT) on labelled examples that the experts never saw. All model selection uses a development split, and scores are reported on the official validation splits. With 256 examples per task, WEMoE reaches an average score of 69.9 ± 0.1, against 57.4 for Task Arithmetic, 78.2 for the experts and 45.8 for a majority-class predictor, which is 59.1% of the gain between the majority class and the experts. Coefficients that do not depend on the input reach only 58.4 ± 0.0. Upscaling only the first MLP layer gives a similar average (69.1 ± 0.1) with 47.2% of the storage of five experts rather than 73.1%, but the two variants are weak on different tasks. WEMoE matches the experts on no task and is several times slower in batches. Limitations include a single set of experts, small validation sets for two tasks, and development data that exceeds the few-shot budget.

Keywords: model consolidation; weight ensembling; Mixture of Experts; BERT; multi-task learning; few-shot fine-tuning.

Chapter 1. The Problem

1.1 Why One Separate Model per Task Does Not Scale

Pre-trained language models have changed how natural language processing systems are built. An encoder such as BERT [4] can be fine-tuned for a new classification problem with limited data, so a system that supports several language understanding tasks tends to collect one fine-tuned copy of the same architecture per task. Each copy performs well on its own task, yet the collection becomes costly in four ways.

- **Storage.** Every copy holds a complete parameter set, although all copies start from the same pre-trained weights.
- **Training.** Every new task requires another full fine-tuning run, with its own data, compute and model selection.
- **Serving.** A deployment must version, load and route requests to a separate model for each task.
- **Knowledge isolation.** Each model knows only its own task. Whatever it learned during fine-tuning is not available to the others.

1.2 Problem Statement and Research Questions

Let θ_0 be the weights of a pre-trained model, and let θ_i, $i \in \mathcal{T}$, be the weights of models fine-tuned from it for a set $\mathcal{T}$ of T tasks. We call each fine-tuned model an *expert*. The *task vector* of task i is the change that fine-tuning made to the shared starting point:

$$\tau_i = \theta_i - \theta_0. \tag{1.1}$$

The input of the problem is θ_0 with the weights θ_i, or equivalently with the task vectors. The output is one multi-task model with weights $\theta_{\text{mer}} = C(\theta_0, \theta_i)$, which maps a text and its task to a label. The problem is to find a merge function C such that the merged model meets three objectives:

O1. It integrates task knowledge *dynamically*, depending on the input.

O2. It stays close to the quality of the task-specific models.

O3. It needs fewer stored parameters than keeping all the models.

No pass threshold was fixed in advance, so "close" is reported as a measured distance and not as a binary outcome. This study addresses four research questions:

RQ1. How much of the performance lost by static merging can routers recover when they are trained on labelled examples that the experts never saw?

RQ2. How does that recovery depend on the labelled budget?

RQ3. Does the improvement require routing that depends on the input, or would better fixed coefficients be enough?

RQ4. How does the choice of upscaled MLP layers affect per-task behaviour and storage, and what does the method cost at inference?

1.3 Why the Problem Matters and Why It Is Difficult

Model consolidation, the merging of already fine-tuned models into one, matters because many systems serve related language tasks and a shared model makes knowledge reusable across them. Multi-task learning [2] also produces one model, but it trains jointly on all the data and repeats that cost when a task is added. Consolidation starts from models that already exist, so it needs no joint retraining from scratch.

The difficulty lies in how fine-tuned models disagree. Updates that help one task can harm another when they land on the same parameters, which is known as *task interference*. Two models can also pull one weight in opposite directions, so that a sum partly cancels what each of them learned. Any merge must therefore decide how much of each update to keep, and it must do so without the original training data of every task.

1.4 Why Static Merging Alone May Be Insufficient

A static method such as Task Arithmetic [7] combines parameters once and commits to one set of weights for all inputs. The five tasks in this study make the limitation concrete. They differ in what they ask of the encoder: judging one sentence for grammar or sentiment, or relating two sentences through paraphrase, entailment or question answering. A single parameter vector must serve all of these at once, and its scaling factor can only trade them against one another. As Tang et al. [17] argue, the best solution can vary with the input, so one fixed result is unlikely to be the best for every task. The gap this report examines is therefore whether merged weights should change with the input instead of staying fixed.

Chapter 2. The Idea

2.1 Mixture of Experts: The Core Concept

A Mixture of Experts (MoE) [8, 12, 15] solves a problem through several specialised networks $E_1, \ldots, E_n$ and a small network called the *router* or gate, G, which decides how much each expert counts for the current input (Figure 2.1):

$$\mathrm{MoE}(x) = C\Big(\{G_i(x) \cdot E_i(x)\}_{i=1}^{n}\Big), \tag{2.1}$$

where $G_i(x)$ is the contribution of expert i and C is a combination function, classically a sum. Because G takes x as its input, the contributions differ from one input to the next. This is the dynamic behaviour that objective O1 asks for.

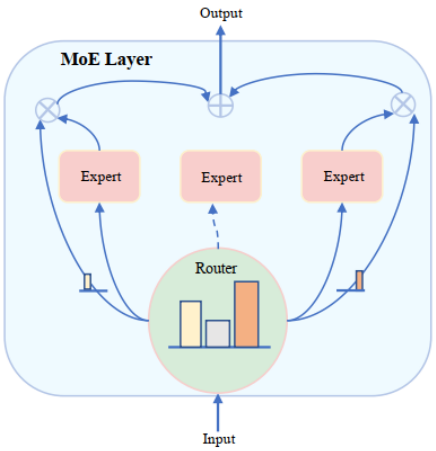


Figure 2.1: A Mixture of Experts layer, from Mu and Lin [12]. The router sets the contribution of each expert for the current input.

2.2 From Output Ensembling to Weight Ensembling

Equation (2.1) combines expert *outputs*. Applied directly, it would run every expert for every input, so inference cost would grow with the number of experts and the experts would need a shared output space. Weight-Ensembling Mixture of Experts (WEMoE) [17] keeps the router but changes what is combined. An expert is represented by its task vector, the router's values scale those vectors, and the resulting weights are assembled before they are applied (Figure 2.2).

The two forms are not equivalent. For a single linear layer, $(\sum_i w_i W_i)h = \sum_i w_i (W_i h)$, but this equality is lost across non-linear activations and stacked layers. Weight ensembling therefore defines a different function from an ensemble of the experts' outputs. Its potential benefits are one forward pass per input, one checkpoint, and a shared encoder. Its costs are that task vectors must stay stored for every upscaled module, that routers need data to train, and that each input needs its own weights assembled at run time, which can make inference slower even when fewer parameters are stored. Section 5.4 measures these costs.

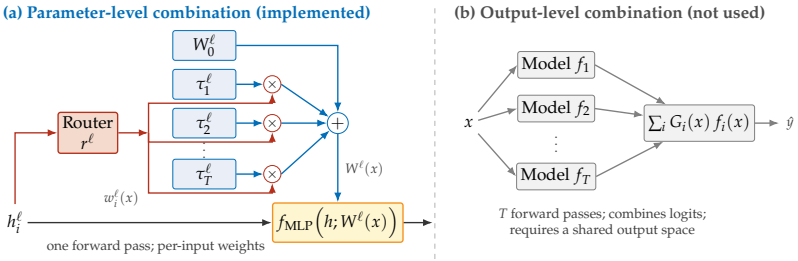


Figure 2.2: Parameter-level combination used in this study (a), contrasted with output-level combination (b), which is shown only for reference.

2.3 Implementation Flow: From Datasets to Evaluation

Figure 2.3 shows the implementation flow in four stages. Stage 1 prepares the five GLUE tasks and divides each official training split into three parts. Stage 2 fine-tunes one expert per task from the pre-trained model, then keeps its task vector and classification head. Stage 3 merges these into one model and trains its routers on the few-shot pool. Stage 4 reports scores on the official validation splits, together with storage and latency. The development split selects the best epoch, the scaling factor λ and the best router step, so no selection touches the reported data. Expert weights stay frozen after Stage 2, and only the routers learn in Stage 3.

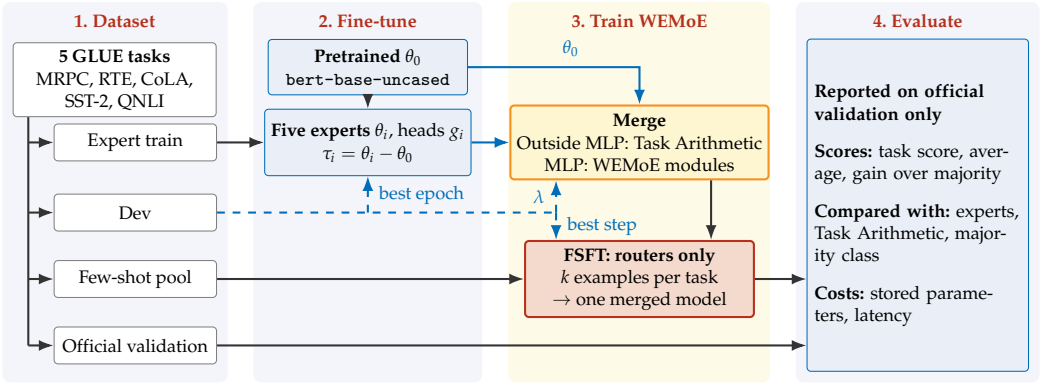


Figure 2.3: Implementation flow: from datasets to evaluation. Dashed arrows show where the development split selects a model. θ_i and τ_i are defined in Section 1.2, g_i is the classification head of expert i, and λ is the Task Arithmetic scaling factor of Section 3.3.

Chapter 3. Theory and Method: WEMoE

Throughout this chapter, $T = 5$ is the number of tasks and experts, i indexes experts, t is the task of an input, $\ell \in \{1, \ldots, 12\}$ indexes encoder layers, and j indexes the N tokens of an input. The method is described as implemented in this study.

3.1 BERT Encoder and Shared Backbone

BERT [4] is an encoder-only Transformer [18]. This study uses `bert-base-uncased`, with 12 encoder layers, hidden size $d = 768$, 12 attention heads and an MLP size of 3072, for 109.5M parameters per expert including the head. Figure 3.1 shows the structure. Tokens are mapped to the sum of a token, a segment and a position embedding, followed by layer normalisation [1]. Each encoder layer then has two sub-layers. Self-attention gives each token a state that reflects its whole context, relating queries Q to keys K of dimension d_k and values V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{3.1}$$

The second sub-layer is a position-wise feed-forward network, called the *MLP block* here:

$$\text{MLP}(h) = W_2\,\text{GELU}(W_1 h + b_1) + b_2, \tag{3.2}$$

where h is a token state, GELU is the activation of Hendrycks and Gimpel [6], and (W_1, b_1) and (W_2, b_2) are the first and second linear layers of the block, called fc1 and fc2. Each sub-layer is wrapped as $h' = \text{LayerNorm}\Big(h + \text{Sublayer}(h)\Big)$. For classification, the final `[CLS]` state passes through the pooler, a dense layer with tanh, and then a task-specific head g_t made of dropout and a linear layer with two outputs. All experts start from the same weights and share this structure, so their weights can be subtracted and added position by position.

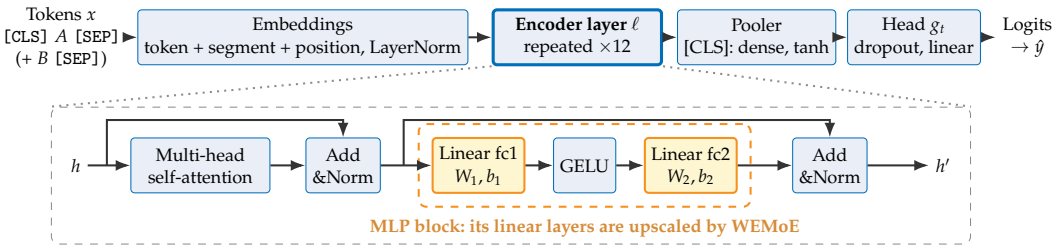


Figure 3.1: BERT encoder as used in this study, from the input tokens to the logits (top), with one encoder layer enlarged (bottom). Dropout layers are not drawn.

3.2 WEMoE Architecture: Static and Input-Conditioned Merging

WEMoE splits the parameters into two groups. In the design of Tang et al. [17], shown in Figure 3.2, attention and layer normalisation are merged statically, and each MLP is "upscaled" to a Weight-Ensembling MoE module, meaning that its weights become input-conditioned. Their vision Transformers normalise before each sub-layer. BERT normalises

after it, so Figure 3.3 shows the same principle as implemented on BERT. Here the static group is every weight of the encoder outside the upscaled layers, including embeddings and pooler. Table 3.1 lists the components, where W_0^ℓ and τ_i^ℓ are the parts of θ_0 and τ_i that belong to the MLP of layer ℓ, λ is the scaling factor of Section 3.3 and r^ℓ is the router of Section 3.4. The router sees only token states and does not receive the task identifier, which selects the head alone.

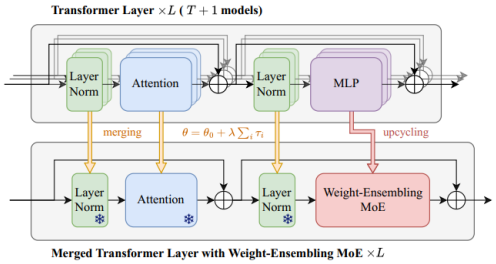


Figure 3.2: Original WEMoE design on a pre-norm vision Transformer, from Tang et al. [17]. Top: the fine-tuned and pre-trained models. Bottom: the merged layer. Frozen parts carry a snowflake mark.

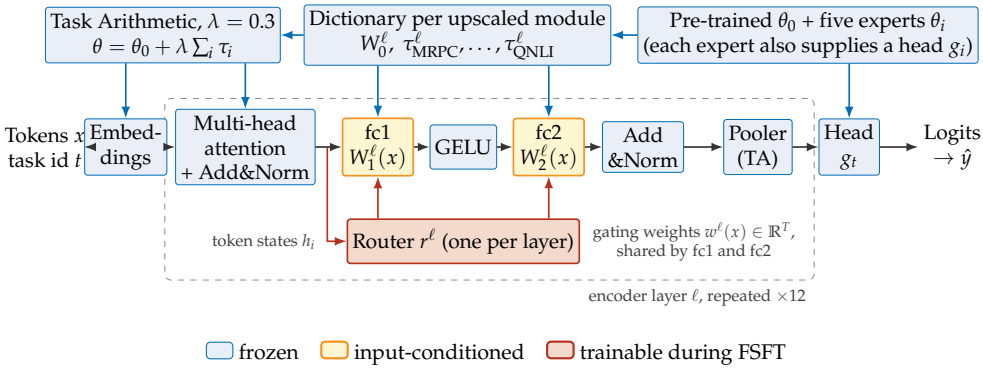


Figure 3.3: Merged WEMoE model as implemented. In the full scope both fc1 and fc2 are input-conditioned. In the intermediate-only scope fc2 is static and uses Task Arithmetic.

Table 3.1: Components of the merged model in the default full-MLP variant.

Component	Origin	During FSFT
Embeddings, attention, layer norm, pooler	Task Arithmetic with λ chosen on dev	Frozen
MLP base weights W_0^ℓ	Pre-trained model	Frozen
MLP task vectors τ_i^ℓ	Experts minus pre-trained model	Frozen
Classification heads g_t	One from each expert	Frozen
Routers r^ℓ	New, one per encoder layer	**Trained**

The split sets the storage cost. A static part is stored once. Each upscaled layer must keep its pre-trained weights and one task vector per expert, which is $T + 1$ copies of that layer, and the routers add their own parameters ϕ:

$$|\theta_{\text{static}}| + (T + 1)\,|\theta_{\text{MLP}}| + |\phi| + |\text{heads}|, \tag{3.3}$$

where θ_{MLP} denotes the upscaled layers. We evaluate two scopes: *full*, which upscales fc1 and fc2, and *intermediate-only*, which upscales fc1 alone. The task vectors account for most of the

stored parameters of the full scope (Table 5.5), so the saving depends strongly on this choice. Tang et al. base the choice of the MLP on their observation for CLIP-ViT models that attention weights stay closer in direction to the pre-trained model than MLP weights do. We did not repeat that measurement on BERT, so the choice is inherited and not independently justified here.

3.3 Task Arithmetic and Task Vectors

A task vector (Equation 1.1) records what fine-tuning changed for a task. Task Arithmetic [7] adds the vectors back to the base with one scaling factor λ:

$$\theta = \theta_0 + \lambda \sum_{i=1}^{T} \tau_i. \tag{3.4}$$

Simple Averaging is the special case $\lambda = 1/T$, which equals the mean of the expert weights. A small λ stays near the pre-trained model, and a large one applies the updates more strongly, including any conflicts among them. We select λ by grid search on the mean development score (Section 4.4). The method is simple and adds no parameters, but its result is the same for every input.

3.4 Router and Input-Conditioned MLP

Each encoder layer has its own router r^ℓ (Figure 3.4). Its input is the sequence of token states $h_j^\ell \in \mathbb{R}^d$ that leave the first Add&Norm of the layer. The router has two linear layers with a ReLU and is applied to every token:

$$r^\ell(h) = A_2^\ell \,\mathrm{ReLU}\left(A_1^\ell h + c_1^\ell\right) + c_2^\ell, \tag{3.5}$$

with $A_1^\ell \in \mathbb{R}^{d\times d}$ and $A_2^\ell \in \mathbb{R}^{T\times d}$. The twelve routers hold $12\,(d^2 + d + Td + T) = 7{,}133{,}244$ trainable parameters. Token outputs are averaged over non-padding positions, using the attention mask $m_j \in \{0, 1\}$, to give one vector per input, so that padding does not affect routing:

$$w^\ell(x) = \frac{\sum_j m_j\, r^\ell(h_j^\ell)}{\sum_j m_j} \in \mathbb{R}^T. \tag{3.6}$$

For an input x, each upscaled linear layer is rebuilt from its pre-trained weights and its part of the task vectors, and its bias is rebuilt in the same way:

$$W^\ell(x) = W_0^\ell + \sum_{i=1}^{T} w_i^\ell(x)\,\tau_i^\ell, \qquad b^\ell(x) = b_0^\ell + \sum_{i=1}^{T} w_i^\ell(x)\,\tau_i^{\ell,b}. \tag{3.7}$$

Both linear layers of a block share the same $w^\ell(x)$. The prediction uses the head g_t of the known task, where $\bar{\theta}$ denotes the statically merged weights:

$$p(y \mid x, t) = \mathrm{softmax}\Big(g_t\Big(\mathrm{pool}(\mathrm{BERT}(x; \bar{\theta}, \{W^\ell(x)\}))\Big)\Big), \quad \hat{y} = \arg\max_y\, p(y \mid x, t). \tag{3.8}$$

The router's values are *unnormalised*. With no softmax and no top-k selection, a value can exceed 1 or be negative, so a router can amplify or subtract a task vector, and the combination

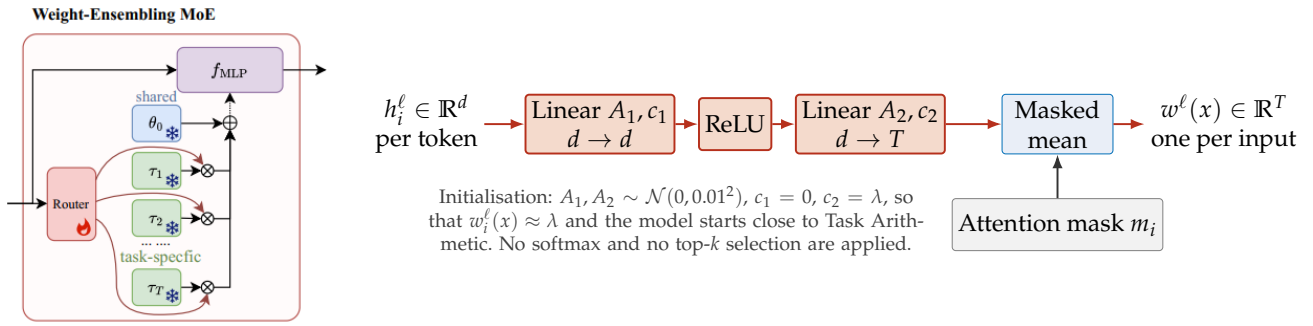


Figure 3.4: Left: the WEMoE module of Tang et al. [17]. Right: router of layer ℓ as implemented, applied per token and averaged over non-padding tokens.

Table 3.2: The original WEMoE study of Tang et al. [17] and this study.

Aspect	Tang et al.	This study
Backbone, tasks	CLIP-ViT vision models; eight image classification tasks	BERT-base; five binary GLUE tasks
Router training	Test-time adaptation: entropy on unlabelled test data	FSFT: cross-entropy on k labelled examples per task
Routing input	Mean over tokens	Mean over non-padding tokens
Upscaled modules	Transformer MLPs	Full MLP; intermediate-only as an ablation

is neither convex nor sparse. Linear weights are initialised from $\mathcal{N}(0, 0.01^2)$ with $c_1^\ell = 0$ and $c_2^\ell = \lambda$, so $w_i^\ell(x) \approx \lambda$ at the start and the model begins close to Task Arithmetic. Unit tests on small random models confirm that constant routing equals Task Arithmetic, that padding does not affect routing and that only the routers receive gradients.

3.5 Few-Shot Fine-Tuning (FSFT) Objective

The routers are trained with Few-Shot Fine-Tuning: cross-entropy on a small labelled set, summed over the tasks. For each task, k examples $D_t^{(k)}$ are drawn from a reserved pool, and for a fixed seed a smaller k gives a subset of a larger one. At each step, one mini-batch B_t is drawn per task, and the objective is

$$\min_{\phi} \sum_{t=1}^{T} \frac{1}{|B_t|} \sum_{(x,y)\in B_t} -\log p_\phi(y \mid x, t), \qquad B_t \subset D_t^{(k)}, \ \ |D_t^{(k)}| = k, \tag{3.9}$$

where p_ϕ is the distribution of Equation (3.8). Everything except ϕ stays frozen, and dropout is disabled. The mean development score is evaluated before training and every 50 steps, and the router state of the best step is restored at the end. A run whose best step is 0 therefore reports the untrained router.

Table 3.2 separates the original method from this study. The largest difference is the objective. Tang et al. minimise prediction entropy on unlabelled test data, which is test-time adaptation, and this objective is not used here. FSFT instead uses labelled training examples and never touches the reporting split. The results of the two studies are therefore not directly comparable.

Chapter 4. Experimental Methodology

4.1 The Five Binary GLUE Tasks and Their Datasets

The experiments use five binary tasks of the General Language Understanding Evaluation benchmark (GLUE) [19], loaded through the Hugging Face Datasets library [9] and listed in Table 4.1: MRPC [5], RTE [3], CoLA [20], SST-2 [16] and QNLI, which is derived from SQuAD [14]. CoLA and SST-2 take a single sentence, while MRPC, RTE and QNLI take a pair. Inputs are tokenised and truncated to 128 tokens. The tasks differ widely in size, as Table 4.2 shows, and in how much a constant prediction scores, as the last column of Table 4.1 shows.

Table 4.1: The five GLUE tasks. "Maj. %" is the validation accuracy of always predicting the most frequent label.

Task	Decides whether…	Input fields	Labels	Metric	Maj. %
MRPC	two sentences are paraphrases	sentence1, sentence2	not_equivalent / equivalent	acc + F1	68.4
RTE	a sentence entails another	sentence1, sentence2	entailment / not_entailment	accuracy	52.7
CoLA	a sentence is grammatical	sentence	unacceptable / acceptable	MCC	69.1
SST-2	a sentence is positive or negative	sentence	negative / positive	accuracy	50.9
QNLI	a sentence answers a question	question, sentence	entailment / not_entailment	accuracy	50.5

4.2 Data Splits and Experimental Protocol

The official training split of each task is permuted once with seed 42 and divided into three disjoint parts (Table 4.2). Four kinds of data therefore have distinct roles:

- **Expert training data** is everything left after the two reservations below. Only the experts train on it.
- **The few-shot pool** holds 256 labelled examples per task, and k of them train the routers. The experts never see it.
- **The development split** (dev) is 10% of the official training split, with at most 1,000 examples. It makes every choice: expert epoch, λ and FSFT step.
- **The official validation split** is used for reported scores only.

GLUE test splits are not used. Their labels are hidden in the dataset for four of the tasks, and the MRPC test labels were also left unused. This protocol prevents the main forms of leakage, at a cost: RTE, the smallest task, keeps only 1,985 of its 2,490 training pairs for its expert.

4.3 Training the Five Task-Specific Experts

Each expert fine-tunes all layers of `bert-base-uncased` and a new two-class head with cross-entropy on its expert-training data. All five use one recipe, shown in the first column of

Table 4.2: Data splits per task. The last row gives the use of each split.

Task	Official train	Few-shot pool	Dev	Expert train	Official validation
MRPC	3,668	256	367	3,045	408
RTE	2,490	256	249	1,985	277
CoLA	8,551	256	855	7,440	1,043
SST-2	67,349	256	1,000	66,093	872
QNLI	104,743	256	1,000	103,487	5,463
Used for		**FSFT**	**Selection**	**Experts**	**Reporting**

Table 4.3, with no per-task tuning, and the best of three epochs is chosen on dev by task score. AdamW [10] applies weight decay only to parameters with at least two dimensions.

Table 4.3: Training configuration of the two optimisation stages.

Setting	Task-specific experts	FSFT (routers)
Trainable parameters	All of BERT and the head (109.5M per expert)	Routers only (7.1M); 60 in the input-independent variant
Optimiser	AdamW; weight decay 0.01	AdamW; no weight decay
Learning rate	2×10^{-5}	10^{-4}; 10^{-3} in the input-independent variant
Schedule	Linear, 10% warm-up	Linear, 10% warm-up
Batch size, duration	32; 3 epochs	16 per task; 500 steps
Gradient clipping	Norm 1.0	None
Selection on dev	Best epoch by task score	Best step by mean score, every 50 steps

4.4 Merging the Experts and Training the Router with Few-Shot Data

Task Arithmetic selects λ from $\{0.1, 0.2, \ldots, 1.0\}$ by the mean dev score (Table 4.4). The score peaks at $\lambda = 0.3$, and this value initialises the merged models. Dev can mislead for merged models on SST-2: at $\lambda = 0.3$, Task Arithmetic scores 83.7 on dev but 69.8 on validation. The two splits differ in kind, since SST-2 training texts average 9.4 words and validation texts 19.5. Selecting on dev remains the correct procedure, but its SST-2 scores overstate merged-model quality.

Table 4.4: Development scores of Task Arithmetic over the λ grid ($\times 100$).

λ	0.1	0.2	**0.3**	0.4	0.5	0.6	0.7	0.8	0.9	1.0
Mean dev score	52.0	57.4	**62.8**	60.7	58.6	58.0	57.5	57.3	57.4	57.1
SST-2 dev score	62.4	73.3	**83.7**	89.8	91.7	92.3	93.0	93.2	92.9	92.5

Fifteen FSFT runs were performed, three seeds for each of five configurations. The main model is *full-MLP* WEMoE with $k \in \{16, 64, 256\}$. Two *ablations*, which change one component and keep the rest fixed, use $k = 256$. The *intermediate-only* variant upscales fc1 alone, and fc2

uses Task Arithmetic. The *input-independent* variant has no router: it learns one coefficient per expert per layer, the same for every input, for 60 numbers in total. It starts exactly at Task Arithmetic and tests whether routing on the input is needed. Its learning rate is higher because it has so few parameters.

4.5 Evaluation Metrics, Baselines, Seeds and Reproducibility

Metrics. Scores are multiplied by 100. The *task score* S_t follows GLUE conventions: the mean of accuracy and F1 for MRPC, the Matthews Correlation Coefficient (MCC) [11] for CoLA, and accuracy for the others. MCC is a correlation between predictions and labels, and a constant prediction scores 0. The *average score* is the unweighted mean of the five task scores, and the *normalised score* is the mean of $S_t(m)/S_t(\text{expert})$ for a model m. A constant prediction already scores about 75 on MRPC and about 50 on the accuracy tasks, so the normalised score flatters weak models. The *gain over majority* corrects for this, with 0 for a constant guess and 100 for the expert:

$$\text{Gain}(m) = \frac{100}{T} \sum_{t=1}^{T} \frac{S_t(m) - S_t(\text{majority})}{S_t(\text{expert}) - S_t(\text{majority})}. \tag{4.1}$$

Baselines. The *majority class* always predicts the most frequent validation label, which is the score of a model collapsed to a constant prediction. *Simple Averaging* is Task Arithmetic with $\lambda = 1/5$. *Task Arithmetic* uses the selected $\lambda = 0.3$. *WEMoE init* is the merged model before router training. The five experts are the upper reference point.

Seeds and reproducibility. The data split and the experts use seed 42. FSFT uses seeds 0, 1 and 2, where a seed sets the router initialisation and the few-shot sample, while the experts and the data split stay fixed. Runs used Google Colab with one NVIDIA Tesla T4 GPU and mixed precision, with PyTorch 2.11.0, Datasets 4.8.5 and scikit-learn 1.6.1. The recorded Transformers [21] version is 5.20.2 for the experts and most FSFT runs, and 5.22.0 for the baseline evaluation and the input-independent runs.

Chapter 5. Results

All scores come from the official validation splits. WEMoE rows give the mean and standard deviation over seeds 0, 1 and 2.

5.1 Results by Task

Table 5.1 and Figure 5.1 give the scores per task, and Table 5.2 places each merged model between the majority class and the expert. The experts themselves are not equally strong. SST-2 and QNLI, with the most training data, score above 90. RTE scores only 63.2, and its dev score was still rising at the last epoch (49.0, 65.5, 65.9). Expert training took 43 minutes in total: 0.8, 0.7, 1.1, 11.0 and 29.8 minutes for MRPC, RTE, CoLA, SST-2 and QNLI, with best epochs 2, 3, 3, 2 and 3.

Table 5.1: Per-task scores (×100). Bold marks the WEMoE $k = 256$ results that fall below the majority class.

Model	MRPC (Acc/F1)	RTE (Acc)	CoLA (MCC)	SST-2 (Acc)	QNLI (Acc)
Majority class	68.4 / 81.2	52.7	0.0	50.9	50.5
Simple Averaging	68.4 / 81.2	46.9	23.3	55.4	65.9
Task Arithmetic	68.6 / 81.3	47.3	16.9	69.8	77.9
Input-independent, k=256	68.4 / 81.2	47.3	20.2±0.0	70.9±0.1	78.6±0.0
WEMoE full, k=16	68.7±2.3 / 80.6±2.2	48.0±1.0	27.6±14.1	79.6±8.1	74.0±6.5
WEMoE full, k=64	71.6±4.4 / 80.9±4.3	54.0±3.4	20.5±17.8	89.0±0.6	83.1±2.4
WEMoE full, k=256	76.3±4.5 / 83.9±2.8	**52.0±4.4**	42.6±0.9	88.0±0.5	86.7±0.7
WEMoE intermediate, k=256	**66.5±0.7 / 76.6±1.0**	59.0±0.9	41.7±0.9	86.7±0.2	86.5±0.2
Single-task experts	82.1 / 88.1	63.2	59.1	92.2	91.2

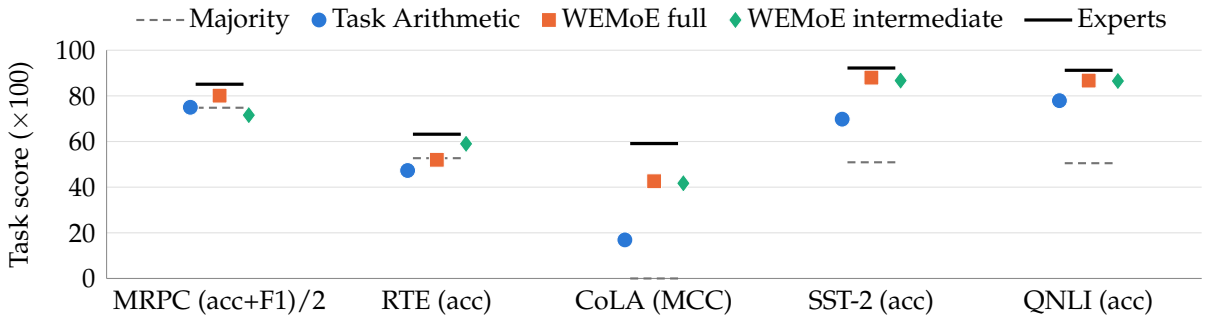


Figure 5.1: Task scores of the merged models between the majority class (dashed tick) and the experts (solid tick). WEMoE values use $k = 256$ and are means over three seeds.

The average improvement does not apply uniformly. On SST-2 and QNLI, the trained routers recover close to 90% of the gain, and on CoLA about 70%, against 45.8, 67.3 and 28.6 for Task Arithmetic. Static merging is close to a constant prediction on the two smallest tasks. Its RTE accuracy of 47.3 equals 131/277, the share of the *minority* class, which is consistent with one label for every example, and its MRPC scores barely differ from the majority class. WEMoE does not fully repair this. The full-MLP model recovers about half of the gain on MRPC but does not reliably beat the majority class on RTE: its three seeds give 49.5, 57.0 and

Table 5.2: Gain over majority per task, in percent, computed from the seed means.

Model	MRPC	RTE	CoLA	SST-2	QNLI	Mean
Task Arithmetic	1.8	−51.7	28.6	45.8	67.3	18.4
Input-independent, $k = 256$	0.0	−51.7	34.2	48.5	68.9	20.0
WEMoE full, $k = 256$	51.5	−6.9	72.2	89.7	88.9	59.1
WEMoE intermediate, $k = 256$	−31.6	59.8	70.5	86.8	88.5	54.8

49.5, against 52.7. The intermediate-only model recovers 59.8% on RTE but falls below the majority class on MRPC. No merged model matches an expert on any task.

5.2 Comparison with Relevant Baselines

Table 5.3 gives the aggregate comparison. Task Arithmetic reaches 57.4 and recovers only 18.4% of the gain, although its normalised score of 70.6 would suggest much more. Before training, the full-MLP model is close to it (57.8 ± 0.6) but not identical, because the small random router weights move CoLA in particular. At $k = 256$, FSFT raises the model to 69.9 ± 0.1, which is 12.5 points above Task Arithmetic and 8.3 below the experts, with a gain of 59.1 ± 1.1.

Table 5.3: Aggregate results ($\times 100$). "Init" is WEMoE before router training. Time covers one FSFT run on the T4, including dev evaluations.

Model	Average	Normalised	Gain	Best step of 500	Min
Majority class	45.8	56.4	0.0	–	–
Simple Averaging	53.3	66.8	6.6	–	–
Task Arithmetic ($\lambda = 0.3$)	57.4	70.6	18.4	–	–
Init, full MLP	57.8±0.6	71.3±1.0	18.7±1.5	–	–
Init, intermediate-only	57.5±0.4	70.8±0.6	18.1±1.2	–	–
Input-independent, $k = 256$	58.4±0.0	72.0±0.0	20.0±0.0	50, 50, 50	15.4
WEMoE full, $k = 16$	60.8±2.7	75.6±4.6	25.5±4.6	50, 0, 150	16.3
WEMoE full, $k = 64$	64.6±3.7	79.5±6.3	46.7±8.5	100, 250, 100	14.4
WEMoE full, $k = 256$	**69.9±0.1**	**87.8±0.5**	**59.1±1.1**	250, 200, 200	16.3
WEMoE intermediate, $k = 256$	69.1±0.1	87.4±0.1	54.8±1.7	500, 400, 400	8.4
Single-task experts	78.2	100.0	100.0	–	–

Input-independent ablation. Coefficients fixed for all inputs barely move the score (58.4 ± 0.0, gain 20.0). They start at the dev score of Task Arithmetic (62.8), stay between 62.4 and 63.0 throughout training, and reach their best score at step 50 on every seed. The learned coefficients lie within 0.023 of their starting value of 0.3. Over the same 500 steps, the full-MLP routers start between 62.6 and 62.9 on dev and reach best scores of 72.6, 73.0 and 72.9. Only one learning rate was tried for this variant.

Few-shot budget. Figure 5.2 shows that the average rises with k and has not levelled off at 256. Small budgets vary strongly across seeds. The budget effect is also not monotonic on

every task: CoLA MCC has a mean of 27.6 at $k = 16$ and 20.5 at $k = 64$, with runs spanning 18–44 and 9–41, and then 42.6 ± 0.9 at $k = 256$. For seed 1 at $k = 16$, no step beat the untrained router on dev, so that run reports WEMoE init. At $k = 256$, every task has a standard deviation of at most 0.9 except MRPC and RTE, so the stable average hides seed variation on those two.

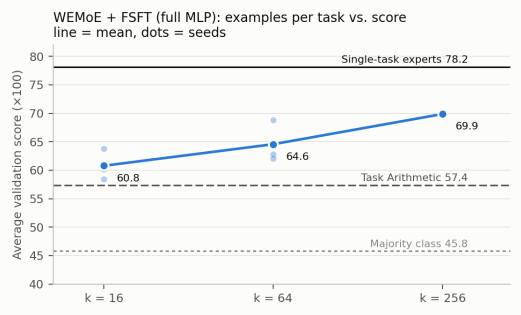


Figure 5.2: Average validation score of full-MLP WEMoE against the few-shot budget. The line shows the mean, and the dots show the seeds.

5.3 Routing Behaviour

Figure 5.3 shows the mean router values per layer and input task for one trained model. Many values lie far from their initial value of about λ, and both signs occur, which a convex mixture could not express. The panels look broadly alike, so much of the pattern is shared across input tasks, although they also differ in places, which shows that routing depends in part on the input. In this model, the column of the RTE expert is negative in the last two layers for every input type.

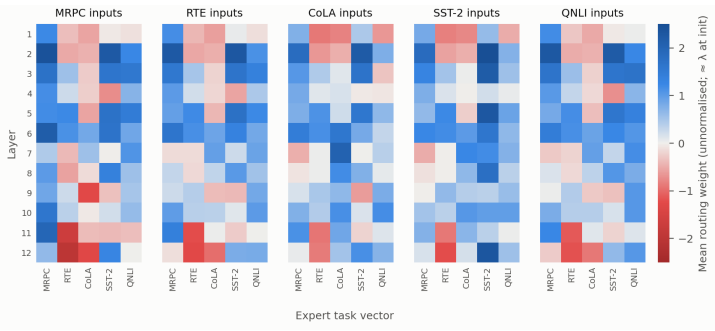


Figure 5.3: Mean router values per layer and input task (full MLP, $k = 256$, seed 0). Blue adds and red subtracts a task vector.

We quantified this pattern from the router values of every validation example. For seed 0, individual values range from -3.0 to 4.4, and the per-layer sum of the values averaged over all examples reaches 5.8, against 1.5 at initialisation. The mean routing matrices of different input tasks correlate at 0.82 on average (0.84 for seeds 1 and 2), while the input task explains about 58% of the routing variance (60% for seeds 1 and 2). Table 5.4 shows that most tasks up-weight their own expert, with RTE as the exception.

The RTE expert behaves differently across scopes, and one seed is not enough to read this difference. Its value on RTE inputs at layer 12 is negative under full-MLP upscaling on all three seeds (-1.28, -0.65, -0.76) and positive under intermediate-only upscaling (2.04, 1.16,

1.53). Its value averaged over layers is less stable: under intermediate-only upscaling it is 0.96, 0.27 and 0.31, while RTE accuracy stays between 58.1 and 59.9, so this average does not explain the RTE result. This evidence shows association only and does not establish why RTE fails.

Table 5.4: Mean router value of a task's own expert, averaged over layers (full MLP, $k = 256$, seed 0).

	MRPC	SST-2	QNLI	CoLA	RTE
On its own inputs	1.37	1.37	0.94	0.61	0.01
On other tasks' inputs	0.77	0.67	0.56	−0.02	−0.01

5.4 Storage, Parameter Count and Inference Latency

Table 5.5 breaks down the stored parameters, which match Equation (3.3) and are measured on the built models. Task vectors dominate, at 283.3M of the 400.0M parameters of the full-MLP model.

Table 5.5: Stored parameters (millions) by component, and total size at 32-bit precision.

Deployment	Static	Base MLP	Task vectors	Routers	Heads	Total	GiB	vs. 5 models
5 single-task models	–	–	–	–	–	547.4	2.04	100.0%
Task Arithmetic	–	–	–	–	–	109.5	0.41	20.0%
WEMoE, full MLP	52.8	56.7	283.3	7.13	0.008	400.0	1.49	73.1%
WEMoE, intermediate-only	81.1	28.3	141.7	7.13	0.008	258.4	0.96	47.2%

Table 5.6 reports latency on one T4 GPU, with random token ids of length 128, mixed precision and the MRPC head, as the mean of 10 forward passes after 3 warm-up passes. The single-task row times one expert, because each input would go to its own expert by task identifier. Batch-1 figures are rough: they differed by up to 27% between two measurement runs (24.05 ms and 18.90 ms for the intermediate-only model), while batch-32 times agreed within about 5%. At batch size 32, the full-MLP model is about 4.5 times slower than one BERT and the intermediate-only model about 2.9 times, and the full-MLP model needs about 2.8 times the peak GPU memory.

Table 5.6: Inference latency and peak GPU memory on one T4, from the final measurement run. *Times one expert.

	Batch 1		Batch 32	
Deployment	ms/example	Peak MB	ms/example	Peak MB
Single-task BERT*	11.64	614	1.61	704
Task Arithmetic	12.67	435	1.63	519
WEMoE, full MLP	27.83	1615	7.31	1987
WEMoE, intermediate-only	18.90	1061	4.69	1387

Chapter 6. Trade-offs and Discussion

6.1 What the Results Demonstrate

The measurements show three things in this setting. Static merging performs close to a constant prediction on the two smallest tasks. Routers trained on 256 examples per task raise the average score by 12.5 points and the gain over majority from 18.4 to 59.1. Fixed coefficients trained on the same data improve the average by only 1 point.

Our interpretation is that input-dependent routing accounts for most of the improvement over Task Arithmetic. This goes beyond the direct measurement, because the two variants also differ in capacity (7.1M against 60 parameters) and in learning rate, and only one learning rate was tried for the ablation, although its best dev score moved by just 0.2.

6.2 Accuracy, Parameter/Storage and Latency Trade-offs

Full-MLP and intermediate-only WEMoE reach similar averages (69.9 and 69.1), but they fail on different tasks. Intermediate-only stores 47.2% of the parameters of five models, is 35% smaller than the full variant, trains in about half the time (8.4 against 16.3 minutes per run) and is faster at inference. Full-MLP has the higher gain (59.1 against 54.8), although the 0.8-point gap in the average is within sampling noise.

Fewer stored parameters do not imply faster inference. Both WEMoE variants store less than five experts, yet they are slower per example, because every example gets its own merged weights and cannot share one matrix multiplication across a batch. Task Arithmetic is the smallest (20%) and matches the speed of one BERT, but it scores 12.5 points below full-MLP WEMoE.

6.3 Practical Implications

Each input in this setup arrives with its task identifier, so the five experts could be selected directly by task. They are more accurate than full-MLP WEMoE on every task, faster per example, and only about 37% larger. When the identifier is known and storage allows it, the experts remain the better choice in this experiment. WEMoE is worth considering mainly when storage must be reduced, and that case is strongest for the intermediate-only variant. Settings without an explicit task identifier may favour it further, but the merged model here still uses the identifier to select the head, and such settings were not evaluated. Averages should also not be read alone, since they conceal task-level failures that the majority-class floor exposes.

6.4 Limitations, Threats to Validity and Unresolved Questions

- **Internal validity.** "Few-shot" is not strict in the sense of Perez et al. [13], because labels used for selection also count: at $k = 256$, FSFT trains on 1,280 labelled examples in total, while selection uses 3,471 dev examples. SST-2 dev scores also overstate merged-model quality (Section 4.4).

- **Statistical validity.** Standard deviations cover three FSFT seeds and exclude variation in the experts and the data split, which use one seed. RTE and MRPC have small validation sets, with standard errors of about 3 and 2 points, larger than several differences discussed here, including 52.0 against 52.7 on RTE.
- **External validity.** The results come from one full run on five binary tasks with one backbone, and they should not be generalised beyond this setup. Backbone, tasks, objective and data all differ from those of Tang et al. [17], so any comparison with that study is qualitative.
- **Construct validity.** The average score mixes accuracy, F1 and MCC. Latency uses random inputs on one GPU, and batch-1 figures varied between runs. Routing statistics show association only, and some of them vary across seeds.

Unresolved questions include why RTE fails under full-MLP upscaling, what the routing variation within a task follows, and how far the score rises beyond $k = 256$.

6.5 Demonstrated Findings and Open Hypotheses

Table 6.1 separates what the measurements establish in this setting from what remains to be tested.

Table 6.1: Demonstrated findings and open hypotheses.

Demonstrated in this setting	Open hypothesis
FSFT with routers improves clearly on Task Arithmetic on average.	Input dependence, rather than capacity, explains most of this improvement.
Fixed coefficients with the tested settings barely improve on it.	A different learning rate would not change this outcome.
Small budgets are unstable; the average still rises at $k = 256$.	Larger budgets would keep raising the score.
The two scopes are weak on different tasks.	The RTE failure comes from trade-offs in the summed objective and mean-score selection, or from a weak RTE expert.
Intermediate-only reaches its best step late (400–500 of 500).	More steps would improve the intermediate-only model.

Chapter 7. Conclusion and Future Work

7.1 Answers to the Research Questions

RQ1. With 256 labelled examples per task, routers raise the average score from 57.4 to 69.9 $\pm$ 0.1 and the gain over majority from 18.4% to 59.1%. A clear gap to the experts (78.2) remains.

RQ2. The average rises from 60.8 at $k = 16$ to 64.6 at $k = 64$ and 69.9 at $k = 256$, and it has not levelled off. Small budgets are unstable across seeds.

RQ3. Fixed coefficients reach only 58.4, so better static coefficients alone do not explain the improvement, although the ablation does not separate input dependence from capacity.

RQ4. Full-MLP and intermediate-only upscaling reach similar averages with different failures, on RTE and on MRPC respectively. Intermediate-only needs 47.2% of the storage of five models, against 73.1%. At batch size 32, the two are about 2.9 and 4.5 times slower than one BERT.

7.2 Assessment of the Objectives

Table 7.1 assesses the objectives of Section 1.2 against the evidence.

Table 7.1: The objectives of the problem statement against the results.

Objective	Evidence	Assessment
O1. Dynamic integration	Per-input MLP weights; fixed coefficients reach 58.4, against 69.9	Met by design; the contribution of input dependence is supported but not isolated
O2. Close to the task-specific models	69.9 against 78.2, gain 59.1%; RTE and MRPC remain weak; no task reaches its expert	Partly met
O3. Fewer stored parameters	73.1% (full MLP) or 47.2% (intermediate-only) of five models	Met, at a cost in latency

7.3 Main Contributions and Practical Implications

The contribution is practical, and no novel algorithm is claimed. This study adapts WEMoE to the post-norm BERT encoder with attention-mask-aware routing, applies a protocol that separates the data of each stage, and evaluates two upscaling scopes, three labelled budgets and an input-independent ablation against a majority-class floor. In practice, input-conditioned merging clearly beats the static merge on average but not the task-specific experts, and its case rests mainly on storage constraints.

7.4 Future Work

Merging four tasks without RTE would test whether RTE interferes with the other tasks. Larger budgets ($k = 512$ and 1024) would show whether the score levels off, and more FSFT steps would test the intermediate-only variant. Measuring task-vector norms per module type would check the reason for upscaling the MLP on BERT. A strict protocol would cap dev at k, and more seeds for the experts and the data split would give more reliable estimates. Per-example analysis would show what routing variation within a task follows.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.

[2] Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997.

[3] Ido Dagan, Oren Glickman, and Bernardo Magnini. The PASCAL recognising textual entailment challenge. In *Machine Learning Challenges*, volume 3944 of *Lecture Notes in Computer Science*, pages 177–190. Springer, 2006.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019. arXiv:1810.04805.

[5] William B. Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP)*, 2005.

[6] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). arXiv preprint arXiv:1606.08415, 2016.

[7] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2212.04089.

[8] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. doi: 10.1162/neco.1991.3.1.79.

[9] Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, et al. Datasets: A community library for natural language processing. In *Proceedings of EMNLP: System Demonstrations*, 2021. arXiv:2109.02846.

[10] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[11] Brian W. Matthews. Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta (BBA) – Protein Structure*, 405(2):442–451, 1975.

[12] Siyuan Mu and Sen Lin. A comprehensive survey of mixture-of-experts: Algorithms, theory, and applications. arXiv preprint arXiv:2503.07137, 2025.

[13] Ethan Perez, Douwe Kiela, and Kyunghyun Cho. True few-shot learning with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2105.11447.

[14] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of EMNLP*, pages 2383–2392, 2016.

[15] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538, 2017.

[16] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of EMNLP*, pages 1631–1642, 2013.

[17] Anke Tang, Li Shen, Yong Luo, Nan Yin, Lefei Zhang, and Dacheng Tao. Merging multi-task models via weight-ensembling mixture of experts. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.00433.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762.

[19] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1804.07461.

[20] Alex Warstadt, Amanpreet Singh, and Samuel R. Bowman. Neural network acceptability judgments. *Transactions of the Association for Computational Linguistics*, 7:625–641, 2019.

[21] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, 2020. arXiv:1910.03771.